Assignment 6: Process State Diagram, Schedulers, and Context Switch

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.1, p. 103 and §3.2, pp. 105–110 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.2, p. 111 and §3.4, p. 135 [3].

6.1 Conceptual

Q1. A classmate claims: "When the disk interrupt arrives, the waiting process starts running at once; and since a ready process might also want to read the disk, `ready` → `waiting` is a perfectly good transition" [2].

(a) Draw the five-state diagram and label every arrow with its triggering event.

(b) For each of the six legal transitions, name the agent that carries it out (a scheduler, the dispatcher, an interrupt handler, or the process itself through a system call).

(c) Refute both parts of the classmate's claim. Explain why separating "wake up" from "get the CPU" is what lets a scheduler enforce a policy.

(d) Explain why a process on a device queue is never on the ready queue at the same instant, using the linked-list implementation of the queues.

Q2. The three schedulers work on different timescales [2, 3].

(a) For each of the long-term, short-term, and medium-term schedulers, state what it moves, how often it runs, and what it controls.

(b) A batch centre admits only I/O-bound jobs for a week, then only CPU-bound jobs the next week. Describe the state of the ready queue, the device queues, the CPU, and the devices in each week, and state which scheduler should have prevented it.

(c) A Linux workstation has no long-term scheduler. Explain what happens when users start far more programs than memory can hold, and which mechanism takes over.

(d) Distinguish a mode switch from a context switch. For each of `getpid`, a timer interrupt at the end of a time slice, a keyboard interrupt that wakes a waiting process, and `read` on an empty pipe, state which occurs and why.

6.2 Numerical

Q3. A context switch costs $s = 4$ μs and each process runs for a slice q before it is switched out.

(a) Compute the overhead fraction $s/(q + s)$ for $q = 2\,\text{ms}$, $q = 200$ μs, and $q = 40$ μs, as percentages to two decimal places.

(b) For $q = 2\,\text{ms}$, compute the approximate number of context switches per second.

(c) Find the smallest slice q for which the overhead is at most 1%.

(d) A short-term scheduler additionally needs $d = 16$ μs to make each decision, and the decision is made once per slice. Write the combined overhead fraction and evaluate it for $q = 200$ μs.

Q4. A one-core machine runs three processes with a 5 ms time slice and negligible switch cost. Ready-queue order is first-in, first-out; a preempted or woken process joins the tail. At $t = 0$, `P1` is running and the ready queue is `P2`, `P3`. The following events occur:

$t = 5$	`P1`'s slice expires
$t = 6$	the running process issues a disk read
$t = 9$	the running process issues a keyboard read
$t = 11$	the disk read completes
$t = 14$	the running process's slice expires
$t = 15$	the running process calls `exit`

(a) Give a table with one row per event showing the running process, the ready queue, the disk queue, and the keyboard queue *after* the event.

(b) List every state transition of every process, with its time.

(c) How many context switches occur? Which event causes a kernel entry but no context switch?

(d) Give the state of each process at $t = 15.5$.

6.3 Design

Q5. A system can hold three process images in memory. At one instant: `P1` is `running`, `P2` is `blocked` on the disk (expected to finish in 8 ms), `P3` is `blocked` on the keyboard (a human may take seconds), and `P4` is `ready/suspend` on disk [3].

(a) `P1` now blocks on the network. Explain why the CPU idles even though `P4` could run.

(b) Design the medium-term scheduler's response: which process do you swap out to make room for `P4`, and why? Give the exact seven-state transitions for the swapped-out process and for `P4`.

(c) While the process you chose is suspended, its awaited event occurs. Give its transition, and explain why it does not move straight to `ready`.

Concepts That Click

(d) State one cost of swapping that the medium-term scheduler must weigh against the benefit in (b).

Q6. You are designing the context-save area of a PCB for a 64-bit CPU with 16 general-purpose registers (8 bytes each), a `PC` (8 bytes), a flags register (8 bytes), a page-table base register (8 bytes), and 32 floating-point registers (16 bytes each) [1].

(a) Give a table of the fields, their sizes, and the total size of the save area in bytes.

(b) Each 8-byte load or store costs 1 ns. Compute the time to save one context and restore another.

(c) Design a "lazy floating-point" variant that saves the floating-point registers only for processes that used them. Recompute (b) for a switch between two processes that never use floating point.

(d) With the full save/restore of (b) and a 1 ms slice, compute the direct switching overhead $s/(q+s)$. Explain why the real cost is higher than this figure.

(e) A CPU offers four hardware register sets. Explain how a switch changes when the next process's context is already in one of the sets, and what must happen when it is not.

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel.* O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts.* John Wiley and Sons, 2018.

[3] William Stallings. *Operating Systems: Internals and Design Principles.* Prentice Hall, 2001.

Concepts That Click